RISC-V Floating-Point Single-Precision Addition Approximation ISA Extension

Ismael Diaz, Zane Alexander Mannings

***Abstract*—Floating-point addition remains a major bottleneck in modern processors due to costly IEEE 754 normalization, rounding, and carry-propagation logic. Prior work shows that floating-point units (FPUs) can consume up to 70–80% of FPGA resources and require up to nine pipeline stages to meet timing, limiting their performance and energy efficiency [2]. Many edge, DSP, and machine-learning workloads, however, can tolerate small arithmetic errors. This motivates approximate computing approaches that intentionally relax accuracy to improve latency and power.**

In this project, we design, implement, and evaluate an approximate single-precision floating-point adder based on the Approximate Floating-Point Precise Carry Prediction Adder (AFPCPA) [3]. Our design partitions the mantissa into precise, imprecise, and truncated regions, reducing critical-path delay while preserving accuracy in the most significant bits. We integrate this adder into a RISC-V processor as a custom instruction (`FADDFAST`) within the gem5 simulator and evaluate microarchitectural performance using cycle-accurate microbenchmarks.

FPGA implementation on a Xilinx Artix-7 device demonstrates frequency improvements comparable to our Vivado timing analysis, reaching up to 534 MHz for low-precision configurations. Error behavior scales exponentially with the number of precise bits, achieving $< 1\%$ average relative error beyond 12 precise mantissa bits. gem5 experiments show an average speedup of 1.33$\times$ across a suite of floating-point microbenchmarks, with significantly reduced latency compared to IEEE 754 addition.

Our results show that approximate floating-point addition offers a compelling design point for constrained systems where performance or power outweighs strict IEEE-754 compliance.

***Index Terms*—Floating-point, approximate units, FPGAs.**

I. Introduction and Problem Statement

PERFORMANCE demands for small-scale, always-on IoT devices continue to grow each year. These devices frequently operate on extremely constrained power budgets, often coin-cell batteries, yet are expected to sustain continuous sensing, inference, and signal-processing workloads. In many practical settings, IoT platforms must compute several million multiply–accumulate operations per second while still maintaining multi-year battery life [8].

Historically, Koomey's Law predicted that the energy required for a fixed amount of computation would halve approximately every 1.5 years. However, this trend has slowed to roughly 2.5 years, indicating that energy-efficiency gains in mobile and embedded processors are no longer keeping pace with application demands. As a result, architectural inefficiencies that were once masked by rapid power scaling have become primary obstacles for always-on computing. One domain where these constraints are particularly evident is floating-point computation. On FPGA-based systems, floating-point units (FPUs) can consume 70–80% of total logic resources, significantly limiting the amount of parallel computation available to the rest of the system [2]. Even basic IEEE 754 single-precision addition involves exponent alignment, mantissa shifts, normalization, rounding, and exception handling. Hemmert and Underwood demonstrate that achieving a modest 270 MHz frequency on FPGA requires a nine-stage pipeline due to strict IEEE 754 compliance. Moreover, further pipeline deepening yields only marginal frequency improvement (approximately 12.5%) while incurring substantial area and power overhead.

At the same time, many workloads, including DSP pipelines, audio and image processing, and machine learning inference, are resilient to minor numerical errors. Small deviations in arithmetic precision often produce negligible perceptual or statistical impact on final results. This observation has fueled the growth of *approximate computing*, where controlled reductions in accuracy are traded for significant improvements in latency, area, and energy efficiency.

Recent architectures such as the Approximate Carry Prediction Adder (ACPA) show that mantissa addition can be simplified by replacing portions of the carry-propagation logic with predictive or approximate circuits, substantially reducing delay while preserving correctness in the most significant bits [3]. These results suggest that strict IEEE 754 adherence may be unnecessarily expensive for error-tolerant applications, and that approximate floating-point addition may offer a compelling design point for low-power, resource-constrained systems.

This project is motivated by the goal of redesigning a floating-point adder to be smaller, faster, and acceptable error by relaxing unneeded precision in the IEEE 754 mantissa computation.

II. Related Work

Research on accelerating floating-point arithmetic spans several domains including approximate computing, normalization simplification, speculative addition, and pipeline reorganization. Much of this work is motivated by the observation that IEEE 754 floating-point addition is dominated by exponent alignment, mantissa addition, and normalization—all of which incur significant latency and hardware cost [2]. Several researchers have therefore explored alternative datapath structures that relax accuracy or re-order arithmetic stages to improve performance. Alexandridis et al. propose an approximate normalization technique for floating-point multiply-add units, reducing shifter complexity and lowering hardware cost while maintaining accuracy suitable for matrix

computation workloads [4]. Their work demonstrates that strict IEEE 754 normalization is often unnecessary for many high-level applications, suggesting that targeted approximations can yield energy-efficient implementations without sacrificing usability.

Sridhar and Kanhe introduce the Approximate Floating-Point Precise Carry Prediction Adder (AFPCPA), a mantissa adder that replaces portions of the traditional full-adder chain with OR–AND approximate cells and a carry prediction circuit [3]. Their approach preserves accuracy in the most significant bits while reducing delay and hardware utilization by 6.68% on average compared to accurate adders. The AFPCPA achieves favorable error metrics (MAE, NMED, MSE) relative to other approximate architectures, making it a compelling candidate for floating-point acceleration in error-tolerant domains.

Beyond AFPCPA, numerous approximate adders have been explored in the literature. Lower-part OR adders (LOA) and their variants simplify the lower bits of addition using logic OR operations, significantly reducing critical-path delay at the expense of higher absolute error. Hybrid Error-Reduction Adders (HERLOA) and Optimized Lower-Part Constant-OR Adders (OLOCA) improve accuracy using selective constant injection and refined carry masking. Other families, such as the Error-Tolerant Adder (ETA) and speculative carry select adders, accelerate addition by predicting or suppressing carries across sub-block boundaries, trading precision for speed and energy savings. These architectures highlight a consistent trade-off: greater simplification of the carry chain yields lower delay but can increase worst-case error.

Researchers have also proposed similar techniques to reduce floating-point latency without approximation. Beaumont-Smith et al. introduce *flagged prefix addition*, merging rounding logic with significand addition to achieve a three-cycle pipelined IEEE 754-compliant adder [6]. Similarly, Hemmert and Underwood reorganize floating-point datapaths to exploit FPGA primitives such as fast carry chains, demonstrating that careful tuning of pipeline depth is required to balance area and performance [2]. These works emphasize that even precise adders require considerable engineering effort to meet timing constraints in reconfigurable logic.

III. Research Questions

This project focuses on understanding both the hardware-level and system-level impacts of introducing approximate floating-point arithmetic into a RISC-V processor. We investigate the following four questions:

1) **How can the hardware cost and delay of IEEE 754 floating-point addition be reduced?** Prior work identifies exponent alignment, mantissa carry propagation, and normalization as the primary contributors to critical-path delay [2]. We examine whether selective approximation of these operations can meaningfully reduce latency while preserving acceptable accuracy.
2) **Which stages of IEEE 754 single-precision addition are most suitable for approximation?** Understanding the datapath structure clarifies where mantissa truncation, OR–AND logic, or predictive carry generation may be safely introduced without destabilizing upper-bit accuracy.
3) **What performance gains arise from replacing a precise adder with an approximate one?** We assess both isolated timing improvements from FPGA implementation and full-system speedups using `gem5` microbenchmarks to quantify latency reductions.
4) **What are the cost, performance, and error trade-offs of varying the number of precise, imprecise, and truncated mantissa bits?** This parametrized design space allows us to map how accuracy loss scales relative to improvements in critical-path delay, maximum clock frequency, and area.

IV. Hypothesis/Proposed Solution

A. Overview

Our hypothesis is that relaxing parts of IEEE 754 compliance, specifically lower-order mantissa accuracy and some correctness guarantees, enables a substantially faster and smaller floating-point adder appropriate for IoT and error-tolerant workloads. Specifically, we believe that replacing the precise mantissa adder with an approximate, carry-predicting adder will significantly reduce critical-path delay. Additionally, parameterizing the mantissa into precise, imprecise, and truncated regions will allow tuning the latency-accuracy trade-off for specific workloads. Eliminating sticky-bit logic, guard/round-bit propagation, and early exception handling will reduce cycle count enough to approach a 3-cycle pipeline similar to Beaumont-Smith's reduced-latency architecture [6]. Finally, integrating this approximate adder as the backend of a RISC-V `FADDFAST` instruction will improve microarchitectural throughput in gem5 simulations without unacceptable numerical error.

In summary, our solution implements a parameterizable approximate floating-point adder in Verilog, integrates it as a functional unit in the RISC-V MinorCPU model in `gem5`, and evaluates its accuracy, performance, and FPGA timing.

B. Approximate Mantissa Adder: AFPCPA Integration

At the core of our proposed design is the Approximate Floating-Point Precise Carry Prediction Adder (AFPCPA) introduced by Sridhar and Kanhe [3] as seen in Figure 1. Unlike conventional adders, which propagate carry across the entire mantissa, AFPCPA splits the mantissa into three regions:

1) **Precise Region (MSBs):** Implemented as a ripple carry adder. This region ensures that high-order bits, the bits most influential on numerical magnitude, remain accurate.
2) **Imprecise Region (Middle Bits):** Implemented using OR–AND approximate full-adder cells. These eliminate long carry chains by locally computing:

$$S = A \vee B \vee C_{in}, \qquad C = A \cdot B$$

which dramatically reduces delay.
3) **Truncated Region (LSBs):** Dropped entirely, as their contribution to the final numerical value is minimal in many DSP and ML workloads.

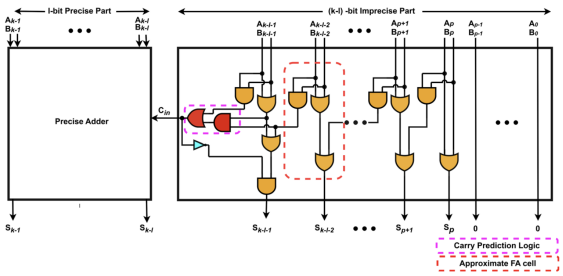

Fig. 1. Architecture of the AFPCPA showing the precise, imprecise, and truncated mantissa regions, along with the carry prediction logic. Reproduced from Sridhar and Kanhe (2024) [3].

A novel carry prediction circuit connects the imprecise and precise regions. This logic predicts the carry that should be forwarded into the precise region, improving accuracy without restoring full carry propagation. Analytical results from [3] show that this carry prediction mechanism reduces MAE, NMED, and MSE by more than 50% compared to other approximate architectures, while requiring fewer hardware resources.

This structure is fully parameterized in our Verilog implementation, allowing:

- selection of the number of precise bits (l),
- imprecise bits ($k - l - p$),
- truncation bits (p),

providing a tunable interface for exploring accuracy-latency trade-offs.

C. Cycle Reduction and Simplified IEEE-754 Path

Achieving a reduced cycle count is a central design goal. Traditional IEEE-754 adders require 6-10 stages due to normalization, rounding modes, sticky-bit tracking, and exception logic. Inspired by the 3-cycle Beaumont-Smith adder pipeline [6], our architecture collapses these stages by:

- **Removing sticky-bit and guard/round-bit logic**: These components are essential for correct IEEE rounding but introduce multi-stage dependencies.
- **Eliminating denormal handling and early-exception paths**: Removing these significantly shortens the combinational logic between registers.
- **Using a single normalization stage**: A simplified left/right shifter replaces multi-stage LZA-based normalization.
- **Merging subtraction, inversion, and sign handling into one stage**: Based on the AFPCPA block structure, sign-mismatch handling is integrated with mantissa alignment.
- **Delegating IEEE conformance to software**: This avoids costly hardware rounding-mode logic while still enabling functionality for workloads that do not depend on IEEE reproducibility.

This results in a three-stage pipelined floating-point adder, as can be seen in Figure 2:

1) **Stage 1:** Operand unpacking, exponent difference, shifting, subtraction/inversion logic.
2) **Stage 2:** Approximate mantissa addition using the AFPCPA (precise + imprecise + truncated).
3) **Stage 3:** Normalization and result assembly (S, E, M).

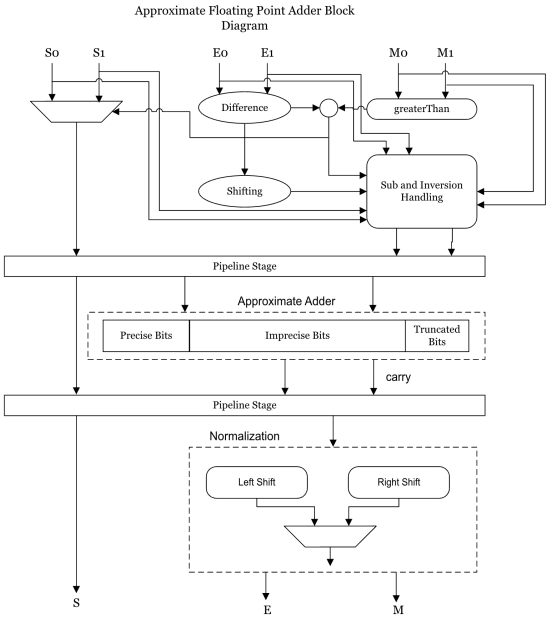

Fig. 2. Approximate Floating-Point Adder block diagram.

By reducing the total number of pipeline stages and removing the longest combinational paths, we hypothesize that this design will operate at significantly higher frequencies than a standard IEEE-754 adder, and thus increase CPU throughput when synthesized into the RISC-V pipeline.

V. Evaluation

The evaluation of our approximate floating-point adder focuses on four primary dimensions: (1) functional correctness and numerical error, (2) physical characteristics measured through FPGA implementation, (3) performance and cost trade-offs as precision is varied, and (4) system-level behavior when integrated into a RISC-V core via gem5 simulation. This mixed-methods approach enables us to capture both the circuit-level implications of approximation and its architectural impact in a realistic processor pipeline.

A. Methodology

Our evaluation proceeds in three stages:

1) **Hardware-Level Assessment (Verilog + Vivado)** The AFPCPA-based floating-point adder is implemented in Verilog and synthesized using Vivado on a Xilinx Artix-7 FPGA. Vivado provides timing estimates, area utilization (LUTs, FFs), and power consumption for different parameterizations of the approximate mantissa. These implementation results form the basis for calculating the critical-path frequency used in gem5 simulations.
2) **Functional Validation and Error Measurement** Cycle-accurate Verilog testbenches evaluate correctness across randomly generated operand pairs. Errors are

compared against a golden IEEE-754 single-precision software model, enabling computation of error metrics.

3) **System-Level Evaluation (gem5)** The approximate adder is incorporated into the RISC-V MinorCPU via the custom `FADDFAST` instruction. Performance is measured using floating-point microbenchmark workloads to quantify CPI, and instruction latency under different approximation configurations.

This pipeline mirrors the structure of Sridhar and Kanhe's AFPCPA evaluation at the circuit level [3] while extending it to full CPU-level performance analysis.

For all evaluation data, configurations consisted of four truncated bits and varying ratios of precise and imprecise bits summing to the twenty remaining bits. For simplicity, configurations are labeled with the number of precise bits.

B. Functional Validation

Functional correctness is validated using a suite of Verilog testbenches that exercise each component of the floating-point datapath:

- exponent difference and alignment logic,
- subtraction and inversion paths,
- approximate and precise mantissa regions,
- normalization and post-processing logic.

For each randomly generated operand pair, both the approximate and IEEE-754 results are captured.

Vivado behavioral simulation additionally provides waveform traces for cycle-by-cycle verification. These traces confirm correct handling of operand unpacking, mantissa shifts, and propagation of the predicted carry from the imprecise region into the precise region of the AFPCPA.

Error statistics as seen in Table 1 and Figure 3 reveal clear trends such as increasing the number of precise mantissa bits dramatically decreases error, approximation errors grow exponentially as the precise region shrinks, truncation errors dominate when p becomes large.

These findings align with analytical predictions from the AFPCPA architecture [3] and help guide optimal parameter selection.

TABLE I
ERROR VS PRECISE BIT CONFIGURATION

Number of Precise Bits	MAE	MSE
2	1.00E+02	5.32E+00
4	1.28E+01	1.38E+00
6	6.31E-01	3.43E-01
8	9.11E-02	8.86E-02
10	2.73E-03	2.13E-02
12	4.51E-04	5.57E-03
14	8.19E-06	1.29E-03
16	4.14E-07	3.11E-04
18	1.90E-08	8.12E-05

C. Vivado Implementation and Physical Characteristics

Vivado implementation provides quantitative insight into the power, frequency, and area trade-offs of different adder configurations. As expected:

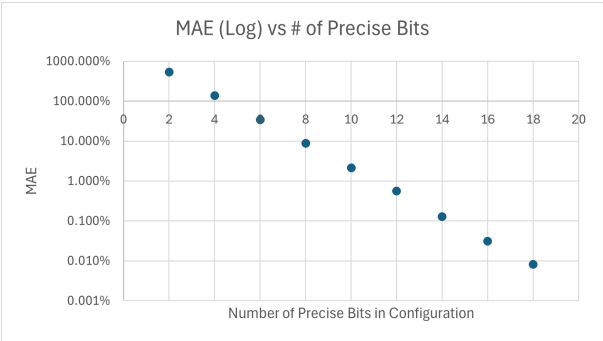

Fig. 3. MAE vs Precise Bits

- **Frequency increases** in Table II as the imprecise and truncated regions grow, reducing carry-chain length. In Figure 4, we can compare that frequency to a 12 cycle and 3 cycle baseline from AMD's Floating Point Operand IP.

TABLE II
VIVADO IMPLEMENTATION RESULTS: CRITICAL PATH

Precise Bits	Logic Depth	Critical Path Latency (ns)	Max Freq. (MHz)
2	2	1.87	534
4	2	2.01	497
6	4	2.48	403
8	5	2.63	380
10	6	2.96	337
12	7	3.03	330
14	8	3.02	331
16	8	3.35	298
18	10	3.81	262

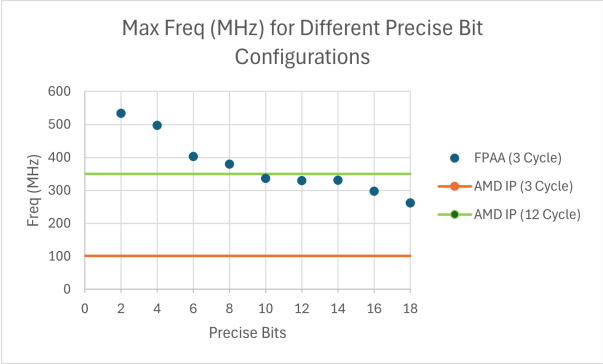

Fig. 4. Max Frequency vs Precise Bits

- **Throughput to Power efficiency decreases modestly with precision**, since higher precision requires more power per clock cycle. The relation between them is illustrated in Table III and Figure 5.

Because our design does not implement sticky bits, guard bits, or IEEE exception signaling, the physical footprint is significantly smaller than the 9-stage IEEE-compliant architecture described by Hemmert and Underwood [2]. Our simplified 3-stage pipeline therefore reaches higher achievable clock frequencies under identical FPGA resource constraints.

TABLE III
VIVADO IMPLEMENTATION RESULTS: POWER

Precise Bits	Power Usage (mW)	Throughput/mW
2	111	4.81
4	109	4.55
6	105	3.83
8	104	3.65
10	103	3.27
12	104	3.17
14	104	3.18
16	102	2.92
18	99	2.64

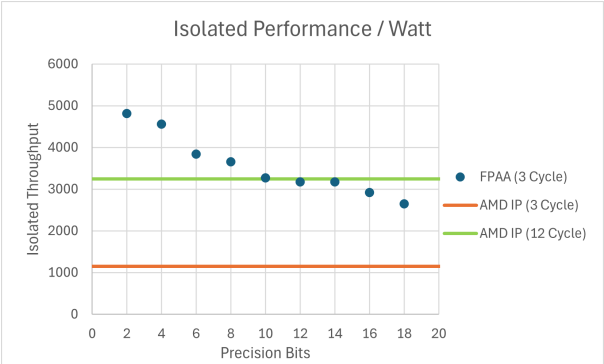

Fig. 5. Throughput/MW vs Precise Bits

Zooming in on area, a cumulative weighted area is calculated to combine the LUTs, FFs, and DSP units used by weighting each element by the total number available in the Artix-7 (ex. Double the number of FFs compared to LUTs mean FFs have half the weight in terms of area usage). The area data can be seen in Table IV.

TABLE IV
VIVADO IMPLEMENTATION RESULTS: AREA

Precise Bits	LUTs (Weight 1)	FFs (Weight 0.5)	Weighted Area
2	349	83	390.5
4	351	84	393
6	356	83	397.5
8	359	84	401
10	349	89	393.5
12	361	84	403
14	357	86	400
16	375	85	417.5
18	352	84	394

Using these results, the maximum stable operating frequency is fed back into the gem5 model to set the timing of the `FADDFAST` instruction. In this way, gem5 observes realistic timing consequences of the approximation.

D. Performance/Cost Tradeoff and Error Discussion

Approximation introduces two interacting effects:

1) **A reduction in latency and hardware cost**, enabling higher clock frequency and lower area.
2) **A degradation in numerical accuracy**, which increases sharply as more low-order bits become imprecise or truncated.

During validation we observed extremely large relative errors (50,000%) under highly aggressive truncation settings. These configurations effectively behave like fixed-point adders and are included only to demonstrate the pathological end of the design space.

More moderate configurations, such as 10–12 precise bits, strike a far more practical balance. They maintain sub-1% average relative error while delivering substantial area and frequency improvements. This matches expectations from the AFPCPA design, where predicted carry into the precise region preserves high-order correctness even when lower-order logic is approximate [3].

The takeaway is that approximation must be tuned: too little yields minimal benefit, while too much produces instability or loss of significance. The parameterized Verilog design allows selecting an approximation level appropriate to the application domain (DSP, ML inference, TinyML, etc.).

E. gem5 Microbenchmarking Results

To measure system-level impact, the approximate adder is integrated into gem5 and evaluated across built-in floating-point microbenchmarks and custome accumulation-heavy workloads. Performance is reported in terms of CPI. Because the frequency of the simulated CPU core is tied to the critical path of the synthesized Verilog unit, more approximate configurations directly increase simulated CPU throughput. Across microbenchmarks, the `FADDFAST` instruction consistently reduced floating-point addition latency and improved CPI by a 1.33 speedup which can also be seen in Figure 3. This also confirms that architectural-level performance benefits track the reductions observed at the circuit level.

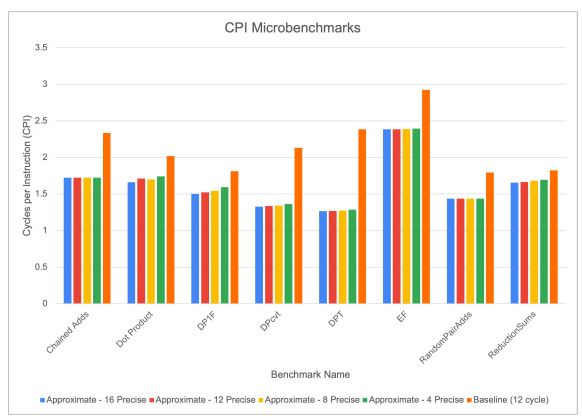

Fig. 6. Mircobenchmark results for CPI (lower is better)

F. Integration as a RISC-V ISA Extension

To evaluate architectural impact, the approximate adder is integrated into the RISC-V MinorCPU model within `gem5` as a new instruction:

$$\texttt{FADDFAST}\ \ rd, rs1, rs2$$

This instruction routes operands to the AFPCPA unit, completely bypassing the precise IEEE-754 unit. Once the synthesized timing is known, its maximum frequency determines the

simulated CPU clock rate in gem5. This allows performance to be measured using floating-point microbenchmarks, enabling analysis of CPI.

Overall, the ISA extension provides a clean architectural interface for exposing approximate hardware to software, and our results show that such an approach can meaningfully improve performance for error-tolerant workloads.

VI. Discussion

Some results await further discussion. Interestingly, the area of the device does not see a significant change depending on the precision bits. There is a slight positive correlation between the number of precision bits and the area, but it alone does not explain the benefits of frequency. Instead, the change of Logic Depth reflects the elimination of carry-chains when replacing precise and imprecise bits. This means that the propagation delay, not the sheer number of gates, is the primary bottleneck within floating-point units.

For power usage, the total power is inversely proportional to the number of precise bits, illustrating the importance of system frequency when comparing power adjusted throughput. In under clocked systems, the Throughput/mW benefit degrades.

It is important to mention the differences in FPGAs and ASICs and the effects on this paper. Frequency and power consumption may see across the board benefits on custom silicone rather than through the Artix-7. Additionally, some of the area benefits of the imprecise adder may have been masked as LUT are discrete blocks.

Lastly, we note that our system-level evaluation was limited due to being unable to validate the design against more extensive SPEC benchmarks. Recompiling SPEC for RISC-V required an OS-specific compiler environment that was incompatible with our EC2 instance. Future work should focus on resolving these compilation issues to verify performance impacts on larger workloads.

VII. Conclusion

In this work, we successfully designed, implemented and evaluated an approximate single-precision floating-point adder based on the AFPCPA architecture. By relaxing strict IEEE-754 compliance on accuracy, we reduced the traditional floating-point pipeline from nine stages to three.

Our FPGA implementation results on the Xilinx Artix-7 demonstrate that different configurations of precise and imprecise bits can significantly reduce critical path latency, achieving operating frequencies up to 534MHz for low-precision configurations. Using these results in gem5, our 3-cycle implementation showed an average speedup of 1.33x across floating-point microbenchmarks. Future work should focus on a greater depth benchmarking results with a measure of how error propagates through the program.

Ultimately, our results show that approximate floating-point addition offers compelling tradeoffs for error-tolerant systems. In domains such as Digital Signal Processing and Machine Learning inference, this module provides a viable path toward higher throughput and efficiency.

References

[1] IEEE Standards Association. (2019). *IEEE standard for floating-point arithmetic* (IEEE Std 754-2019). IEEE Computer Society.

[2] Hemmert, K. S., & Underwood, K. D. (2010). Fast, efficient floating-point adders and multipliers for FPGAs. *ACM Transactions on Reconfigurable Technology and Systems, 3*(3), Article 11. https://doi.org/10.1145/1839480.1839481

[3] Sridhar, C., & Kanhe, A. (2024). Approximate floating point precise carry prediction adder for FIR filter applications. *Circuits, Systems, and Signal Processing, 43*(11), 6487–6509. https://doi.org/10.1007/s00034-024-02760-9

[4] Alexandridis, K., Peltekis, C., Filippas, D., & Dimitrakopoulos, G. (2024). Floating-point multiply-add with approximate normalization for low-cost matrix engines. *arXiv preprint arXiv:2408.11997*. https://arxiv.org/abs/2408.11997

[5] Bhattacharjya, R., Mishra, V., Singh, S., Goswami, K., & Banerjee, D. S. (2020). An approximate carry estimating simultaneous adder with rectification. In *Proceedings of the 30th ACM Great Lakes Symposium on VLSI (GLSVLSI)*. arXiv:2008.11591.

[6] Beaumont-Smith, A., Burgess, N., Lefrere, S., & Lim, C. C. (1999). Reduced latency IEEE floating-point standard adder architectures. In *Proceedings of the 14th IEEE Symposium on Computer Arithmetic (ARITH-14)* (pp. 232–239). IEEE. https://doi.org/10.1109/ARITH.1999.762842

[7] Koomey, J. G., Berard, S., Sanchez, M., & Wong, H. (2011). Implications of historical trends in the electrical efficiency of computing. *IEEE Annals of the History of Computing, 33*(3), 46–54. https://doi.org/10.1109/MAHC.2010.28

[8] Banbury, C. R., Reddi, V. J., Lam, M., Fu, W., Fazel, A., Holleman, J., Huang, X., Hurtado, R., Kanter, D., Lokhmotov, A., Patterson, D., Pau, D., Seo, J., Sieracki, J., Thakker, U., Verhelst, M., & Yadav, P. (2021). *Benchmarking TinyML Systems: Challenges and Direction*. arXiv:2003.04821. https://arxiv.org/abs/2003.04821

Appendix A: Timeline and Milestones

Due to a change in project focus halfway through the allotted project time, our milestones followed an accelerated four-week timeline. The major goals and deliverables were:

- **Milestone 1: Approximate Floating-Point Adder Implementation (Week 1)** Developed a AFPCPA approximation IEEE-754-like single-precision floating-point adder in Verilog. Validated correctness using testbenches and used Vivado to confirm timing behavior and pipeline operation.
- **Milestone 2: Vivado Timing and Error Comparison (Week 2)** Modified the approximate adder parameters to collected Vivado implementation results to evaluate timing, area, and pipeline depth reductions.
- **Milestone 3: gem5 Integration and Simulation (Week 3)** Implemented a new RISC-V instruction (`FADDFAST`) and integrated the approximate adder into gem5. Conducted performance evaluation using floating-point microbenchmarks.
- **Milestone 4: Final Evaluation and Reporting (Week 4)** Analyzed accuracy, performance, and power trade-offs across various adder configurations. Prepared the final project report and presentation.

Appendix B: Division of Labor

Both team members contributed to research, design, implementation, and documentation. Responsibilities were divided as follows:

- **Student A** Implemented the core IEEE-754 floating-point modules and the AFPCPA-based approximate adder along with assisting in functional validation. Ran Vivado implementation analysis, including timing, area, and power measurements.
- **Student B** Integrated the approximate adder into gem5 as a custom RISC-V instruction and conducted performance simulations such as the creation of several microbenchmarks.
- **Both Students** Collaborated on system-level evaluation, analysis of approximation trade-offs, and interpretation of results. Jointly prepared the final report, presentation, and all written deliverables.